Practical SAT Solving

Lecture 3

Prof. Dr. Carsten Sinz

Hochschule Karlsruhe, University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik
Lehrgebiet Software-Entwicklung



HornSAT

Horn Formula

A CNF formula is a *Horn formula* if each of its clauses contains at most one positive literal.

Example: $(\overline{x}_1 \vee \overline{x}_7 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (x_1)$

HornSAT

Horn Formula

A CNF formula is a *Horn formula* if each of its clauses contains at most one positive literal.

Example: $(\overline{x}_1 \vee \overline{x}_7 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (x_1)$

Horn Clauses as Implication Rules

Definite clause	$(\overline{x}_1 \vee \overline{x}_7 \vee x_3)$	$x_1 \wedge x_7 \implies x_3$
Fact	(x_1)	$\top \implies x_1$
Goal clause	$(\overline{x}_2 \vee \overline{x}_4)$	$x_2 \wedge x_4 \implies \bot$

- No negations in implication rules
- Definite clauses: "propagation of facts", inference
- Goal clauses: "proof by contradiction"

Solving Horn Formulas

SAT Algorithm for Horn Formulas

- Perform unit propagation on the input formula
- If you resolve $\bot$ then the formula is UNSAT otherwise it is SAT
- Get the solution:
  - Take the assignment after unit propagation as initial partial solution
  - Set the rest of the variables to False

Mixed Horn

Mixed Horn Formula

A CNF formula is Mixed Horn if it contains only binary and Horn clauses.

Example: $(\overline{x}_1 \vee \overline{x}_7 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (x_1 \vee x_5) \wedge (x_3)$

Questions:

- How to solve a Mixed Horn formula?
- How hard is it to solve a Mixed Horn formula?

Mixed Horn Formula

Mixed Horn Complexity

Mixed Horn SAT is NP-complete

Proof:

- We will reduce SAT to Mixed Horn SAT
- For each non-Horn clause $C = (\ell_1 \vee \ell_2 \vee \dots)$ do
  - for each but one positive $\ell_i \in C$ introduce a new variable ℓ'_i
  - replace ℓ_i in C by $\overline{\ell'_i}$
  - add $(\ell'_i \vee \ell_i) \wedge (\overline{\ell'_i} \vee \overline{\ell_i})$ to establish $\ell_i = \overline{\ell'_i}$

Example: $(x_1 \vee \overline{x}_7 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (x_1 \vee x_5) \rightsquigarrow$
$\rightsquigarrow (\overline{x'_1} \vee \overline{x}_7 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (\overline{x'_1} \vee x_5) \wedge (x'_1 \vee x_1) \wedge (\overline{x'_1} \vee \overline{x_1})$

Hidden/Renamable/Disguised Horn

Hidden Horn Formulas

A CNF formula is *Hidden Horn* if it can be made Horn by renaming some of its variables.

Example:

$(x_1 \vee x_2 \vee x_4) \wedge (x_2 \vee \overline{x}_4) \wedge (x_1) \rightsquigarrow (\overline{x}_1 \vee \overline{x}_2 \vee x_4) \wedge (\overline{x}_2 \vee \overline{x}_4) \wedge (\overline{x}_1)$

Questions:

- How to recognize a Hidden Horn formula?
- How hard is it to recognize and solve a Hidden Horn formula?

Recognizing Hidden Horn Formulas

Translate into 2-SAT

Let F be original formula, R_F contains the clause $(l_1 \vee l_2)$ if and only if there is a clause $C \in F$ such that $l_1 \in C$ and $l_2 \in C$.

Example: $F = (x_1 \vee x_2 \vee x_4) \wedge (x_2 \vee \overline{x}_4) \wedge (x_1)$
$R_F = (x_1 \vee x_2) \wedge (x_1 \vee x_4) \wedge (x_2 \vee x_4) \wedge (x_2 \vee \overline{x}_4)$

Recognize Hidden Horn

If R_F is satisfiable, then F is a hidden Horn formula. Furthermore, the satisfying assignment ϕ of R_F identifies the variables to be renamed.

- if $x_i = \mathit{True}$ in ϕ then x_i needs to be renamed to $\overline{x}_i$

SLUR Formulas

Single Look-ahead Unit Resolution - SLUR algorithm

01 **if** $\perp \in$ unit-prop(F) **then return** UNSAT **else return** SLUR(F)

02 **function** SLUR(F)
03 **if** all variables appear in a unit clause **then return** SAT
04 $v =$ select-variable(F)
05 $F_1 =$ unit-prop($F \wedge (v)$)
06 $F_2 =$ unit-prop($F \wedge (\overline{v})$)
07 **if** $\perp \in F_1$ **and** $\perp \in F_2$ **then return** GIVE-UP
08 **if** $\perp \in F_1$ **and** $\perp \notin F_2$ **then** SLUR(F_2)
09 **if** $\perp \notin F_1$ **and** $\perp \in F_2$ **then** SLUR(F_1)
10 **if** $\perp \notin F_1$ **and** $\perp \notin F_2$ **then any of** SLUR(F_1) **or** SLUR(F_2)

A CNF formula F is SLUR if the SLUR algorithm never gives up on for F (regardless of the choices in lines 04 and 10).

SLUR Formulas

Properties of SLUR Formulas[1]

- SLUR formulas are solvable in polynomial time (using the SLUR algorithm)
- SLUR is an umbrella class for polynomially solvable classes – All Horn and Hidden Horn formulas are SLUR formulas
  - Also true for Extended Horn, Balanced, and Propagation Complete formulas[2]
- It is co-NP complete to recognize whether a given CNF is a SLUR formula or not

[1] Cepek et al., 2012, Properties of SLUR Formulae

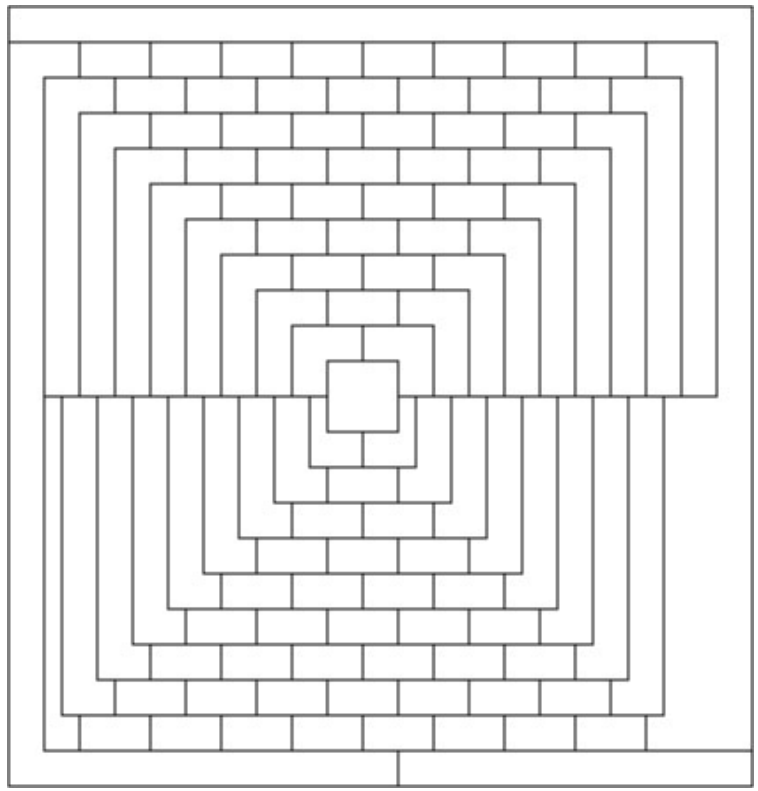
[2] Schlipf et al., 1995, On Finding Solutions for Extended Horn Formulas

Encodings: Motivation

- A wide variety of problems can be encoded as SAT!
  - (Finite) arithmetic
  - Mathematical / practical combinatorial problems
  - Hardware / software verification problems
  - Planning problems
- Chosen encoding highly influences runtime of SAT solver
  - A lot of research on good encodings
  - . . . but still more an art than a science

Cardinality Constraints: Motivation

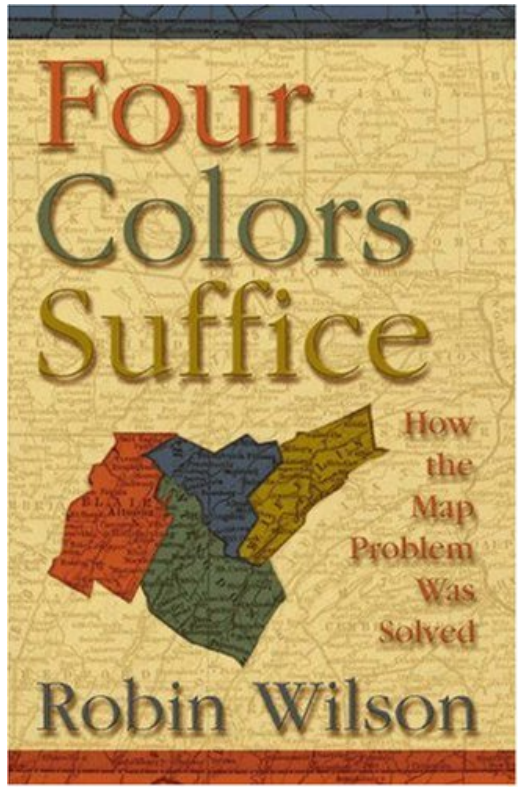
Is there a 4-coloring of the McGregor graph?



Cardinality Constraints: Motivation

Four Color Theorem [Appel, Haken; 1976]

Every planar graph is four-colorable.



Excursion: The Hadwiger-Nelson Prob.

Hadwiger-Nelson-Problem [1950]

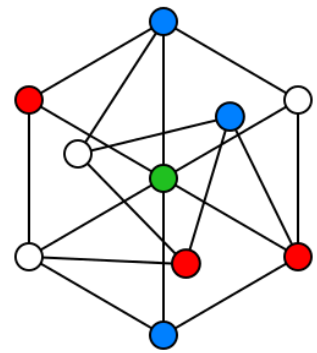
How many colors are needed to color the plane such that no two points at distance 1 from each other have the same color?

Excursion: The Hadwiger-Nelson Prob.

Hadwiger-Nelson-Problem [1950]

How many colors are needed to color the plane such that no two points at distance 1 from each other have the same color?

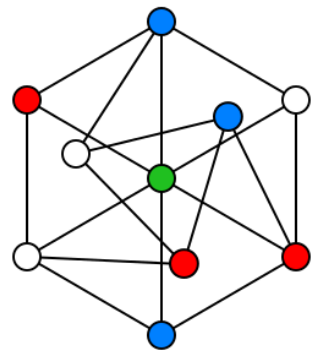
- Known: The result is between 4 and 7



Excursion: The Hadwiger-Nelson Prob.

Hadwiger-Nelson-Problem [1950]

How many colors are needed to color the plane such that no two points at distance 1 from each other have the same color?



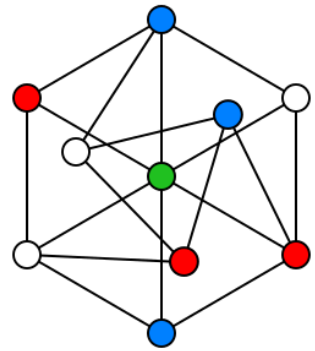
- Known: The result is between 4 and 7
- April 2018: "Amateur mathematician cracks decades-old math problem": Aubrey de Grey found a 1581-vertex, non-4-colourable unit-distance graph



Excursion: The Hadwiger-Nelson Prob.

Hadwiger-Nelson-Problem [1950]

How many colors are needed to color the plane such that no two points at distance 1 from each other have the same color?



- Known: The result is between 4 and 7
- April 2018: "Amateur mathematician cracks decades-old math problem": Aubrey de Grey found a 1581-vertex, non-4-colourable unit-distance graph



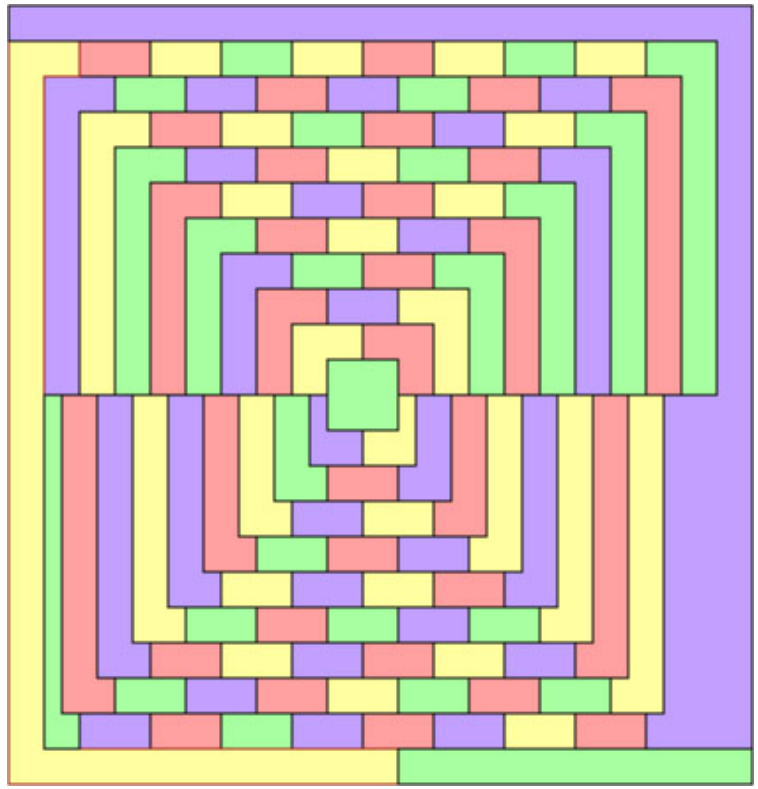
- May 9, 2018: Improved by SAT solver to 633-vertex graph

Cardinality Constraints: Motivation

Is there a 4-coloring of the McGregor graph?

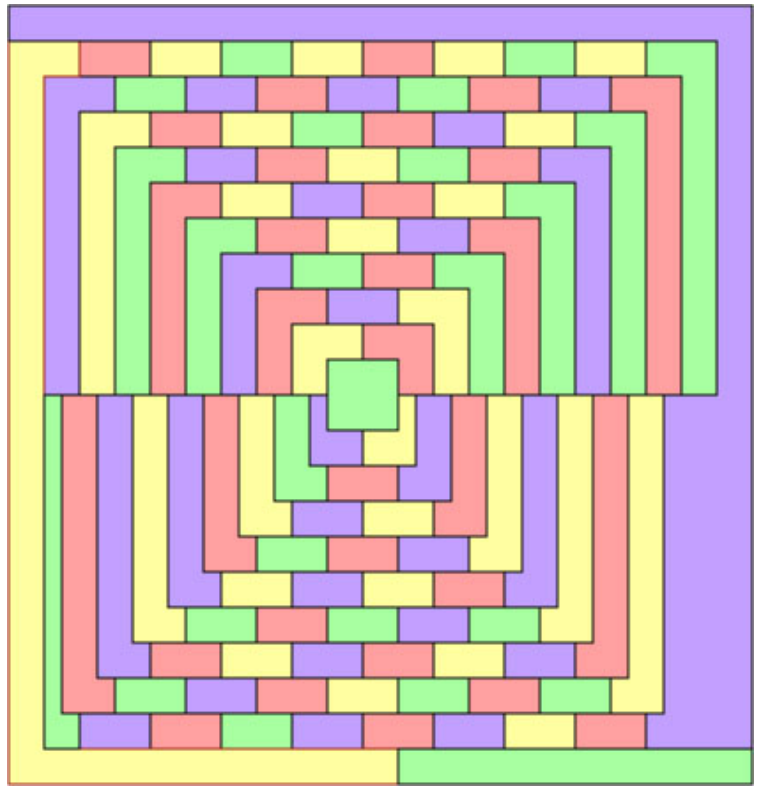
Cardinality Constraints: Motivation

Is there a 4-coloring of the McGregor graph? **Yes!** (see below)



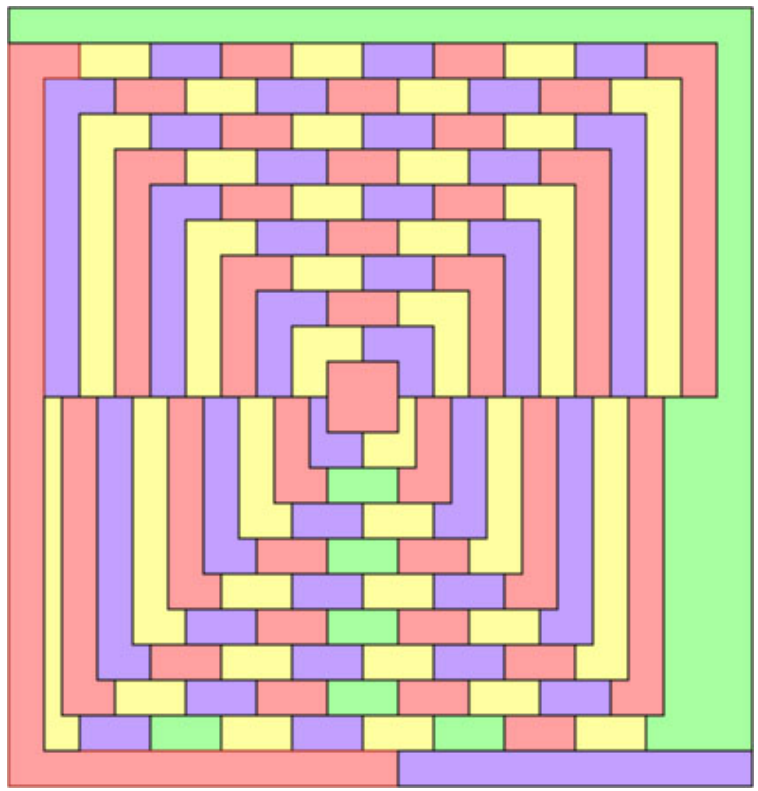
Cardinality Constraints: Motivation

Is there a 4-coloring of the McGregor graph? **Yes!** (see below)
Is there a 4-coloring of the McGregor graph that uses one color at most 7 times?



Cardinality Constraints: Motivation

Is there a 4-coloring of the McGregor graph? **Yes!** (see below)
Is there a 4-coloring of the McGregor graph that uses one color at most 7 times? **Yes!** (see below)



Cardinality Constraints

Definition

$\leq k\,(x_1, \ldots, x_n)$ is the constraint that no more than k variables / literals out of $x_1, \ldots, x_n$ are set to True.

Cardinality Constraints

Definition

$\leq k\,(x_1, \ldots, x_n)$ is the constraint that no more than k variables / literals out of $x_1, \ldots, x_n$ are set to True.

- Naive encoding: add clauses $(\overline{x_{i_1}} \vee \cdots \vee \overline{x_{i_{k+1}}})$ for $1 \leq i_1 < \cdots < i_{k+1} \leq n$
  - Results in $\binom{n}{k+1}$ clauses
  - Worst case ($k = \lceil n/2 \rceil$): $\mathcal{O}(2^n/\sqrt{n})$ clauses (by Sterling's Approximation)

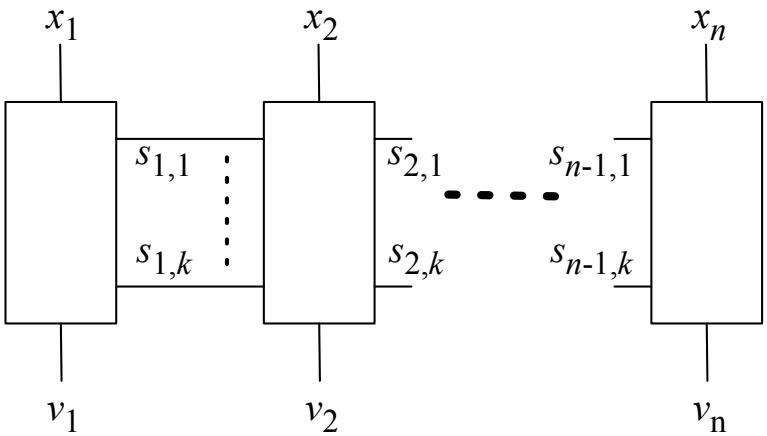
Cardinality Constraints

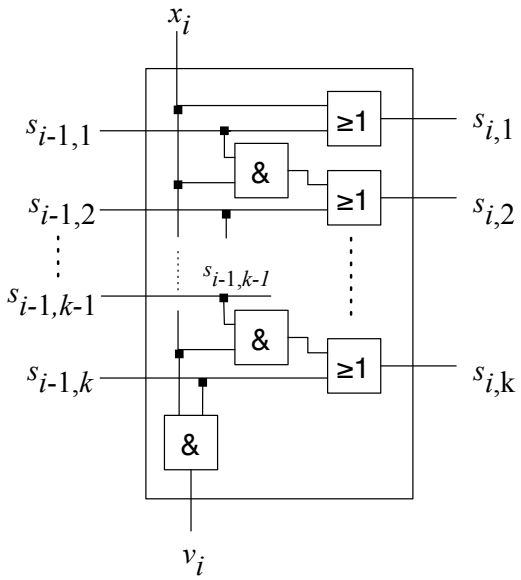
Definition

$\leq k\,(x_1, \ldots, x_n)$ is the constraint that no more than k variables / literals out of $x_1, \ldots, x_n$ are set to True.

- Naive encoding: add clauses $(\overline{x_{i_1}} \vee \cdots \vee \overline{x_{i_{k+1}}})$ for $1 \leq i_1 < \cdots < i_{k+1} \leq n$
  - Results in $\binom{n}{k+1}$ clauses
  - Worst case ($k = \lceil n/2 \rceil$): $\mathcal{O}(2^n/\sqrt{n})$ clauses (by Sterling's Approximation)
- Better encodings [1]:
  - Sequential counter: $\mathcal{O}(n \cdot k)$
  - Parallel counter: $\mathcal{O}(n)$

Cardinality Constr.: Sequential Counter





$$
\begin{array}{l}
(\neg x_1 \vee s_{1,1}) \\
(\neg s_{1,j}) \qquad \text{for } 1 < j \leq k \\
(\neg x_i \vee s_{i,1}) \\
(\neg s_{i-1,1} \vee s_{i,1}) \\
(\neg x_i \vee \neg s_{i-1,j-1} \vee s_{i,j}) \\
(\neg s_{i-1,j} \vee s_{i,j}) \\
(\neg x_i \vee \neg s_{i-1,k}) \\
(\neg x_n \vee \neg s_{n-1,k})
\end{array}
\quad \left.\begin{array}{l} \\ \\ \\ \\ \\ \\ \end{array}\right\} \text{for } 1 < j \leq k \quad \left.\begin{array}{l} \\ \\ \\ \\ \\ \\ \end{array}\right\} \text{for } 1 < i < n
$$

Encoding Circuits

Problem

Given a formula F in propositional logic with operations $\wedge$, $\vee$ and $\neg$, how can it be encoded in CNF?

Encoding Circuits

Problem

Given a formula F in propositional logic with operations $\wedge$, $\vee$ and $\neg$, how can it be encoded in CNF?

Example

$F = \neg((\neg x \vee y) \wedge (\neg z \wedge \neg(x \wedge \neg w)))$

Encoding Circuits

Problem

Given a formula F in propositional logic with operations $\wedge$, $\vee$ and $\neg$, how can it be encoded in CNF?

Example

$F = \neg((\neg x \vee y) \wedge (\neg z \wedge \neg(x \wedge \neg w)))$

First approach: convert to NNF (Negation Normal Form), then apply distributive law.

Encoding Circuits

Problem

Given a formula F in propositional logic with operations $\wedge$, $\vee$ and $\neg$, how can it be encoded in CNF?

Example

$F = \neg((\neg x \vee y) \wedge (\neg z \wedge \neg(x \wedge \neg w)))$

First approach: convert to NNF (Negation Normal Form), then apply distributive law.

Example (cont'd)

$F_{\text{NNF}} = (x \wedge \neg y) \vee z \vee (x \wedge \neg w)$
$F_{\text{CNF}} = (x \vee z) \wedge (x \vee z \vee \neg w) \wedge (\neg y \vee z \vee x) \wedge (\neg y \vee z \vee \neg w)$

Encoding Circuits

Problem

Given a formula F in propositional logic with operations $\wedge$, $\vee$ and $\neg$, how can it be encoded in CNF?

Example

$F = \neg((\neg x \vee y) \wedge (\neg z \wedge \neg(x \wedge \neg w)))$

First approach: convert to NNF (Negation Normal Form), then apply distributive law.

Example (cont'd)

$F_{\text{NNF}} = (x \wedge \neg y) \vee z \vee (x \wedge \neg w)$
$F_{\text{CNF}} = (x \vee z) \wedge (x \vee z \vee \neg w) \wedge (\neg y \vee z \vee x) \wedge (\neg y \vee z \vee \neg w)$

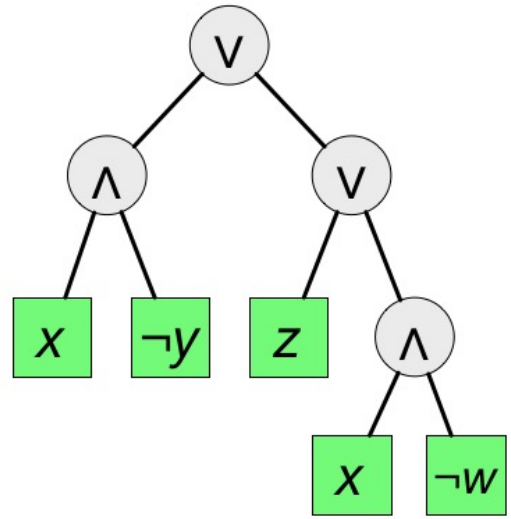
Problem: Applying the distributive law may result in an exponential blow-up.

Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$

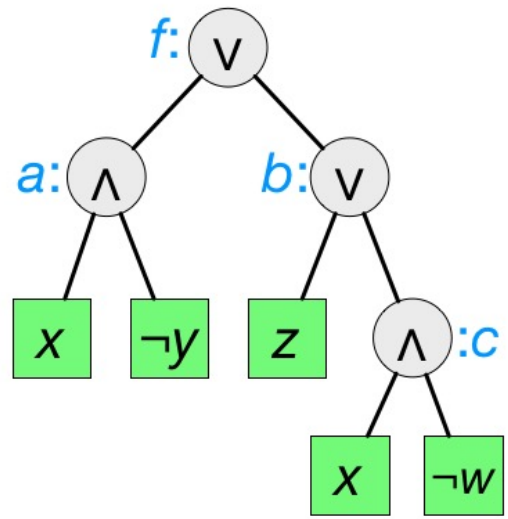


Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$

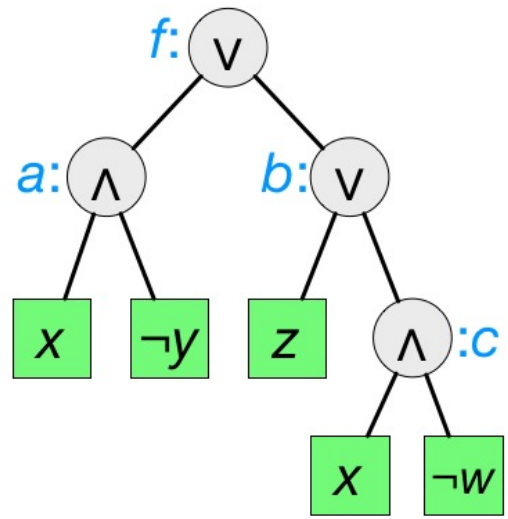


Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$
- "Triplets" using subformulas:
  $f = a \vee b, \quad a = x \wedge \overline{y}, \ldots$

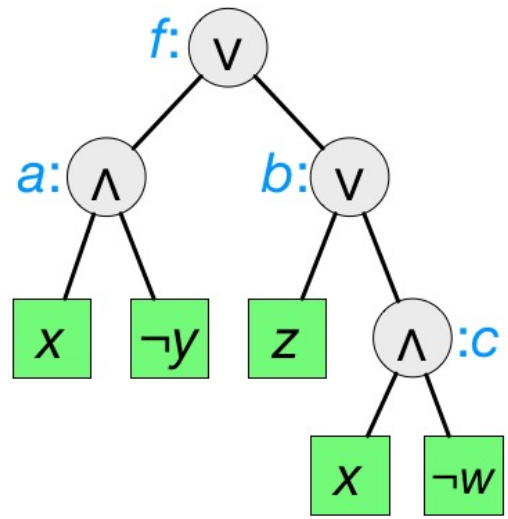


Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$
- "Triplets" using subformulas:
  $f = a \vee b, \quad a = x \wedge \overline{y}, \ldots$
- Encode each triplet (as equivalence) in CNF:
  $(\overline{f} \vee a \vee b) \wedge (f \vee \overline{a}) \wedge (f \vee \overline{b}) \wedge \ldots$

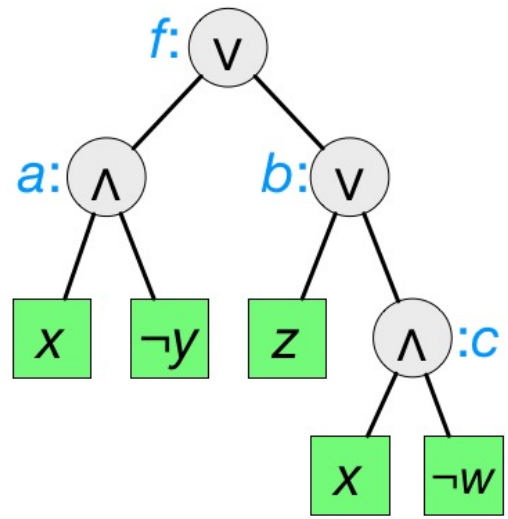


Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$
- "Triplets" using subformulas:
  $f = a \vee b, \quad a = x \wedge \overline{y}, \ldots$
- Encode each triplet (as equivalence) in CNF:
  $(\overline{f} \vee a \vee b) \wedge (f \vee \overline{a}) \wedge (f \vee \overline{b}) \wedge \ldots$
- One additional clause (f) to assert that F must be true

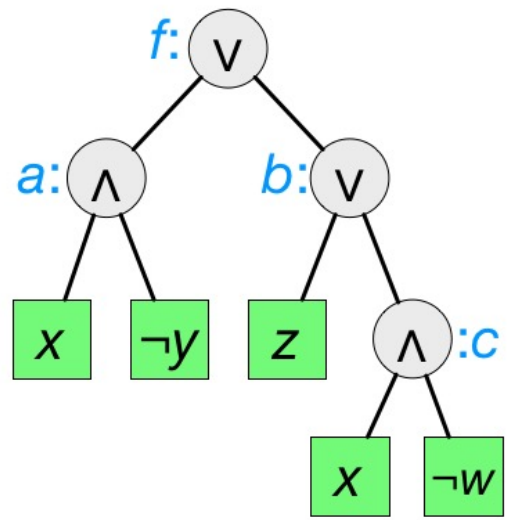


Tseitin Encoding

Idea

Introduce new variables for subformulas.

- Example:
  $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$
- "Triplets" using subformulas:
  $f = a \vee b, \quad a = x \wedge \overline{y}, \ldots$
- Encode each triplet (as equivalence) in CNF:
  $(\overline{f} \vee a \vee b) \wedge (f \vee \overline{a}) \wedge (f \vee \overline{b}) \wedge \ldots$
- One additional clause (f) to assert that F must be true
- Note: Sometimes implication instead of equivalence is sufficient



Tseitin Encoding: Definition

Given a formula F in propositional logic using connectives $\wedge$, $\vee$ and $\neg$. The Tseitin-Encoding $\mathcal{T}(F)$ of F is a formula in CNF defined by:

$$\mathcal{T}(F) = d_F \wedge \mathcal{T}^*(F)$$

$$\mathcal{T}^*(F) = \begin{cases} \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}^*(G) \wedge \mathcal{T}^*(H), & \text{if } F = G \circ H \text{ and } \circ \in \{\wedge, \vee\} \\ \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}^*(G), & \text{if } F = \neg G \\ \mathit{True}, & \text{if } F \in \mathcal{V} \end{cases}$$

$$\mathcal{T}_{\text{def}}(F) = \begin{cases} (\overline{d_F} \vee d_G) \wedge (\overline{d_F} \vee d_H) \wedge (d_F \vee \overline{d_G} \vee \overline{d_H}), & \text{if } F = G \wedge H \\ (\overline{d_F} \vee d_G \vee d_H) \vee (d_F \vee \overline{d_G}) \wedge (d_F \vee \overline{d_H}), & \text{if } F = G \vee H \\ (\overline{d_F} \vee \overline{d_G}) \wedge (d_F \vee d_G), & \text{if } F = \neg G \end{cases}$$

$\mathcal{T}(F)$ introduces a new variable d_S for each subformula S of F.
$\mathcal{T}(F)$ is satisfiable iff F is satisfiable.

Tseitin Encoding: Example

- $F = (x \wedge \neg y) \vee (z \vee (x \wedge \neg w))$

Tseitin Encoding: Example

- $F = \underbrace{\overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}}_{f}$
- Use auxiliary variables for subformulas as indicated above.

Tseitin Encoding: Example

- $F = \underbrace{\overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}}_{f}$
- Use auxiliary variables for subformulas as indicated above.
- To simplify exposition, we treat negative literals like variables in $\mathcal{T}(F)$.

Tseitin Encoding: Example

- $F = \underbrace{\overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}}_{f}$
- Use auxiliary variables for subformulas as indicated above.
- To simplify exposition, we treat negative literals like variables in $\mathcal{T}(F)$.

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}^*(F) \\
&= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}^*(S_a) \wedge \mathcal{T}^*(S_b) \\
&= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= f \wedge \underbrace{(\overline{f} \vee a \vee b) \wedge (f \vee \overline{a}) \wedge (f \vee \overline{b})}_{\mathcal{T}_{\text{def}}(F)} \wedge \cdots
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= f \wedge (\overline{f} \vee a \vee b) \wedge (f \vee \overline{a}) \wedge (f \vee \overline{b}) \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge (a \vee \overline{x} \vee y) \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge (b \vee \overline{z}) \wedge (b \vee \overline{c}) \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge (c \vee \overline{x} \vee w)
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= f \wedge (\overline{f} \vee a \vee b) \wedge \cancel{(f \vee \overline{a})} \wedge \cancel{(f \vee \overline{b})} \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge (a \vee \overline{x} \vee y) \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge (b \vee \overline{z}) \wedge (b \vee \overline{c}) \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge (c \vee \overline{x} \vee w)
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= \cancel{f} \wedge (\overline{f} \vee a \vee b) \wedge \cancel{(f \vee \overline{a})} \wedge \cancel{(f \vee \overline{b})} \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge (a \vee \overline{x} \vee y) \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge (b \vee \overline{z}) \wedge (b \vee \overline{c}) \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge (c \vee \overline{x} \vee w)
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= \cancel{f} \wedge (\overline{f} \vee a \vee b) \wedge \cancel{(f \vee \overline{a})} \wedge \cancel{(f \vee \overline{b})} \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge \cancel{(a \vee \overline{x} \vee y)} \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge (b \vee \overline{z}) \wedge (b \vee \overline{c}) \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge (c \vee \overline{x} \vee w)
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= \cancel{f} \wedge (\overline{f} \vee a \vee b) \wedge \cancel{(f \vee \overline{a})} \wedge \cancel{(f \vee \overline{b})} \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge \cancel{(a \vee \overline{x} \vee y)} \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge \cancel{(b \vee \overline{z})} \wedge \cancel{(b \vee \overline{c})} \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge \cancel{(c \vee \overline{x} \vee w)}
\end{aligned}
$$

Tseitin Encoding: Example (cont'd)

- $F = \overbrace{(x \wedge \neg y)}^{a,\, S_a} \vee \underbrace{(z \vee \overbrace{(x \wedge \neg w)}^{c,\, S_c})}_{b,\, S_b}$

$$
\begin{aligned}
\mathcal{T}(F) &= f \wedge \mathcal{T}_{\text{def}}(F) \wedge \mathcal{T}_{\text{def}}(S_a) \wedge \mathcal{T}_{\text{def}}(S_b) \wedge \mathcal{T}_{\text{def}}(S_c) \\
&= \cancel{f} \wedge (\cancel{\overline{f}} \vee a \vee b) \wedge \cancel{(f \vee \overline{a})} \wedge \cancel{(f \vee \overline{b})} \\
&\quad \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge \cancel{(a \vee \overline{x} \vee y)} \\
&\quad \wedge (\overline{b} \vee z \vee c) \wedge \cancel{(b \vee \overline{z})} \wedge \cancel{(b \vee \overline{c})} \\
&\quad \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w}) \wedge \cancel{(c \vee \overline{x} \vee w)} \\
&\overset{\text{SAT}}{=} (a \vee b) \wedge (\overline{a} \vee x) \wedge (\overline{a} \vee \overline{y}) \wedge (\overline{b} \vee z \vee c) \wedge (\overline{c} \vee x) \wedge (\overline{c} \vee \overline{w})
\end{aligned}
$$

Finite-Domain Variables

- Common in combinatorial problems: finite domain variables, e.g.:
  $x \in \{v_1, \ldots, v_n\}$
- Relationships between them expressed as equality-formulas, e.g.:
  $x = v_3 \Rightarrow y \neq v_2$.

Finite-Domain Variables

- Common in combinatorial problems: finite domain variables, e.g.:
  $x \in \{v_1, \ldots, v_n\}$
- Relationships between them expressed as equality-formulas, e.g.:
  $x = v_3 \Rightarrow y \neq v_2$.
- Direct encoding / "one-hot-encoding":
  - Boolean variables x_v: "x takes value v"
  - Must encode that each variable takes exactly one value from its domain (using at-least-one/at-most-one constraints)
  - Encoding of variables' constraints simple

Finite-Domain Variables

- Common in combinatorial problems: finite domain variables, e.g.:
  $x \in \{v_1, \ldots, v_n\}$
- Relationships between them expressed as equality-formulas, e.g.:
  $x = v_3 \Rightarrow y \neq v_2$.
- Direct encoding / "one-hot-encoding":
  - Boolean variables x_v: "x takes value v"
  - Must encode that each variable takes exactly one value from its domain (using at-least-one/at-most-one constraints)
  - Encoding of variables' constraints simple
- Log-encoding / binary encoding:
  - Boolean variables b_i^x for $0 \leq i < \lceil \log_2 n \rceil$
  - Each value gets assigned a binary number, e.g.
    $v_1 \rightarrow 00, v_2 \rightarrow 01, v_3 \rightarrow 10$
  - Inadmissible values must be excluded, e.g.:
    $x \in \{v_1, v_2, v_3\}$ requires $(\overline{b_0^x} \vee \overline{b_1^x})$
  - Encoding of constraints can become complicated

Comparing Encodings

- Size: number of variables, number of clauses
- Propagation properties

At-Most-One Constraints

Definition

AtMostOne($x_1, \ldots, x_n$) is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

At-Most-One Constraints

Definition

AtMostOne($x_1, \ldots, x_n$) is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

- Alternative notations: $\leq 1\,(x_1, \ldots, x_n)$, $\quad x_1 + \cdots + x_n \leq 1$

At-Most-One Constraints

Definition

AtMostOne$(x_1, \ldots, x_n)$ is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

- Alternative notations: $\leq 1\,(x_1, \ldots, x_n), \quad x_1 + \cdots + x_n \leq 1$
- Naive (pairwise) encoding: add clauses $(\overline{x_i} \vee \overline{x_j})$ for $1 \leq i < j \leq n$
  - Results in $\binom{n}{2} = \frac{n \cdot (n-1)}{2}$ clauses

At-Most-One Constraints

Definition

AtMostOne$(x_1, \ldots, x_n)$ is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

- Alternative notations: $\leq 1\,(x_1, \ldots, x_n), \quad x_1 + \cdots + x_n \leq 1$
- Naive (pairwise) encoding: add clauses $(\overline{x_i} \vee \overline{x_j})$ for $1 \leq i < j \leq n$
  - Results in $\binom{n}{2} = \frac{n \cdot (n-1)}{2}$ clauses
- Can we do better?

At-Most-One Constraints

Definition

AtMostOne($x_1, \ldots, x_n$) is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

- Alternative notations: $\leq 1\,(x_1, \ldots, x_n)$, $\quad x_1 + \cdots + x_n \leq 1$
- Naive (pairwise) encoding: add clauses $(\overline{x_i} \vee \overline{x_j})$ for $1 \leq i < j \leq n$
  - Results in $\binom{n}{2} = \frac{n \cdot (n-1)}{2}$ clauses
- Can we do better? Yes!

At-Most-One Constraints

Definition

AtMostOne($x_1, \ldots, x_n$) is the constraint that no more than 1 variable / literal out of $x_1, \ldots, x_n$ is set to True.

- Alternative notations: $\leq 1\,(x_1, \ldots, x_n)$, $\quad x_1 + \cdots + x_n \leq 1$
- Naive (pairwise) encoding: add clauses $(\overline{x_i} \vee \overline{x_j})$ for $1 \leq i < j \leq n$
  - Results in $\binom{n}{2} = \frac{n \cdot (n-1)}{2}$ clauses
- Can we do better? Yes!
- Encodings and their complexity (in number of clauses):[3]
  - Pairwise encoding: $\mathcal{O}(n^2)$ (without extra variables)
  - Ladder encoding: $\mathcal{O}(n)$ (with n extra variables)
  - Tree encoding: $\mathcal{O}(n \log n)$ (with $\log n$ extra variables)

[3] Hoelldobler, 2013, An Efficient Encoding of the at-most-one Constraint

References I


C. Sinz, Towards an optimal CNF encoding of boolean cardinality constraints, in: Proc. of the 11th Intl. Conf. on Principles and Practice of Constraint Programming (CP 2005), Sitges, Spain, 2005, pp. 827–831.